암호학적 기초 요소

개요

파일코인의 저장 증명 시스템은 여러 암호학적 기초 요소들을 조합하여 안전하고 효율적인 증명 메커니즘을 구현합니다. 이 문서에서는 파일코인에서 사용되는 핵심 암호학적 구성 요소들을 상세히 분석합니다.

해시 함수들

파일코인은 서로 다른 목적과 특성을 가진 세 가지 주요 해시 함수를 사용합니다.

SHA256

특성과 용도:

- **알고리즘**: NIST 표준 SHA-2 계열의 256비트 해시 함수
- **출력 크기**: 32바이트 (256비트)
- **주요 용도**:
- DRG 그래프에서 부모 노드 기반 키 생성
- 레거시 시스템과의 호환성 유지
- 일반적인 데이터 무결성 검증

구현 특징:

```
// SHA256 해시 함수의 기본 사용
fn hash(data: &[u8]) -> Sha256Domain {
```

```
        let hashed = Sha256::digest(data);

        let mut res = Sha256Domain::default();

        res.0.copy_from_slice(&hashed[..]);

        res.trim_to_fr32(); // Fr 필드에 맞게 조정

        res

}
```

- `trim_to_fr32()` 함수를 통해 마지막 2비트를 제거하여 BLS12-381 스칼라 필드에 맞게 조정
- 회로 구현에서는 비트 순서를 빅엔디안에서 리틀엔디안으로 변환

Blake2s

특성과 용도:

- **알고리즘**: BLAKE2s 해시 함수 (SHA-3 경쟁에서 파이널리스트)
- **출력 크기**: 32바이트 (256비트)
- **주요 용도**:
- 고성능이 요구되는 해시 연산
- 머클 트리 구축에서의 노드 해시
- 일부 PoRep 구현에서 사용

성능 특징:

- SHA256보다 빠른 소프트웨어 구현
- 하드웨어 최적화가 용이
- zk-SNARK 회로에서 상대적으로 효율적

Poseidon

특성과 용도:

- **알고리즘**: zk-SNARK 친화적 해시 함수
- **출력 크기**: 32바이트 (BLS12-381 스칼라 필드 원소)
- **주요 용도**:
- zk-SNARK 회로 내부에서의 해시 연산
- 머클 트리 구축 (특히 회로 증명용)
- 현재 파일코인의 주요 해시 함수

zk-SNARK 최적화:

```
// Poseidon 해시의 다양한 arity 지원
match  preimage.len() {
        2 => Poseidon::new_with_preimage(preimage, &POSEIDON_CONSTANTS_2),
        4 => Poseidon::new_with_preimage(preimage, &POSEIDON_CONSTANTS_4),
        8 => Poseidon::new_with_preimage(preimage, &POSEIDON_CONSTANTS_8),
        16 => Poseidon::new_with_preimage(preimage, &POSEIDON_CONSTANTS_16),
        _ => panic!("Unsupported arity"),
}
```

- 2, 4, 8, 16개의 입력을 지원하는 다양한 arity
- 회로 제약 조건 수가 SHA256/Blake2s보다 현저히 적음
- BLS12-381 필드에서 직접 연산 수행

머클 트리 구조

파일코인은 다양한 형태의 머클 트리를 사용하여 데이터 무결성과 증명 생성을 수행합니다.

트리 유형별 분류

Binary Merkle Tree (이진 머클 트리)

- **Arity**: 2 (각 내부 노드가 2개의 자식을 가짐)
- **용도**: 기본적인 데이터 무결성 증명
- **특징**: 가장 단순한 구조, 증명 크기가 $\log_2(n)$

Quad Tree (4진 트리)

- **Arity**: 4 (각 내부 노드가 4개의 자식을 가짐)
- **용도**: 증명 크기와 계산 복잡도의 균형
- **특징**: 증명 크기가 $\log_4(n)$, 해시 연산 횟수 증가

Octary Tree (8진 트리)

- **Arity**: 8 (각 내부 노드가 8개의 자식을 가짐)
- **용도**: 대용량 섹터에서 증명 크기 최소화
- **특징**: 증명 크기가 $\log_8(n)$, Poseidon 해시와 최적화

저장 방식별 분류

DiskTree (디스크 저장 트리)

```
pub  type DiskTree<H, U, V, W> = MerkleTreeWrapper<H, DiskStore<H::Domain>, U, V
```

- **특징**: 모든 노드를 디스크에 저장
- **용도**: 대용량 데이터 처리, 메모리 사용량 최소화

- **장점**: 메모리 효율적, 영구 저장
- **단점**: 디스크 I/O로 인한 성능 저하

LCTree (레벨 캐시 트리)

```
pub  type LCTree<H, U, V, W> = MerkleTreeWrapper<H, LevelCacheStore<H::Domain, F
```

- **특징**: 일부 레벨을 메모리에 캐시
- **용도**: 성능과 메모리 사용량의 균형
- **장점**: 자주 접근하는 상위 레벨의 빠른 접근
- **단점**: 메모리 사용량 증가

머클 트리 활용 방식

TreeC (Column Tree)

- **목적**: 각 레이어의 컬럼별 데이터 무결성 보장
- **구조**: 각 컬럼이 하나의 머클 트리를 형성
- **사용 시점**: PreCommit Phase 2에서 생성

TreeR (Replica Tree)

- **목적**: 최종 복제본 데이터의 무결성 보장
- **구조**: 전체 복제본 데이터를 포함하는 머클 트리
- **사용 시점**: 모든 레이어 인코딩 완료 후 생성

TreeRLast (Last Layer Replica Tree)

- **목적**: 마지막 레이어 데이터의 무결성 보장
- **구조**: 최종 레이어만을 포함하는 머클 트리

- **사용 시점**: Sealing 과정의 마지막 단계

DRG (Depth Robust Graph) 개념과 구현

DRG는 파일코인 PoRep의 핵심 그래프 구조로, 데이터의 순차적 접근을 강제하여 병렬 공격을 방어합니다.

DRG의 수학적 배경

깊이 견고성 (Depth Robustness)

- **정의**: 그래프에서 임의의 노드 집합을 제거해도 남은 그래프의 깊이가 유지되는 성질
- **목적**: 공격자가 일부 데이터를 삭제하더라도 전체 데이터 재생성에 필요한 계산량 보장
- **수학적 표현**: δ-깊이 견고 그래프는 최대 δ개의 노드를 제거해도 깊이가 유지됨

그래프 매개변수

```
pub const BASE_DEGREE: usize = 6; // 기본 차수
```

- **노드 수**: 섹터 크기에 따라 결정 (32바이트 단위)
- **차수**: 각 노드가 가지는 부모 노드의 수 (6개)
- **깊이**: 그래프의 최대 경로 길이

BucketGraph 샘플링 알고리즘

BucketGraph는 DRG의 구체적인 구현으로, 효율적인 부모 노드 선택 알고리즘을 사용합니다.

샘플링 과정

1. **메타그래프 노드 계산**:

```
let m_prime = m - 1; // m은 차수
let metagraph_node = node as u64 * m_prime as u64;
```

2. **버킷 수 결정**:

```
let n_buckets = (metagraph_node as f64).log2().ceil() as u64;
```

3. **버킷별 샘플링**:

```
for parent in other_drg_parents.iter_mut().take(m_prime) {
        let bucket_index = (rng.gen::<u64>() % n_buckets) + 1;
        let largest_distance_in_bucket = min(metagraph_node, 1 << bucket_index)
        let smallest_distance_in_bucket = max(2, largest_distance_in_bucket >>
        let n_distances_in_bucket = largest_distance_in_bucket - smallest_dista
        let distance = smallest_distance_in_bucket + (rng.gen::<u64>() % n_dist
        let metagraph_parent = metagraph_node - distance;
        let mapped_parent = (metagraph_parent / m_prime as u64) as u32;
        *parent = if mapped_parent == node { node - 1 } else { mapped_parent };
}
```

알고리즘 특징

버킷 기반 샘플링:

- 각 노드의 부모를 선택할 때 거리에 따라 버킷을 나눔
- 가까운 거리의 버킷일수록 높은 확률로 선택

- 로그 분포를 따라 부모 노드 분산

시드 기반 결정론적 생성:

```
let mut seed = [0u8; 32];
seed[..28].copy_from_slice(&self.seed);
seed[28..].copy_from_slice(&node.to_le_bytes());
let mut rng = ChaCha8Rng::from_seed(seed);
```

- PoRep ID에서 파생된 시드 사용
- 노드별로 고유한 시드 생성
- ChaCha8 PRNG를 통한 안전한 난수 생성

즉시 전임자 보장:

```
parents[predecessor_index] = node - 1;
```

- 각 노드는 반드시 바로 이전 노드를 부모로 가짐
- 순차적 계산 강제를 통한 병렬화 방지
- API 버전에 따라 전임자 위치 조정

Stacked DRG 확장

Stacked DRG는 기본 DRG에 확장 부모(expansion parents)를 추가한 구조입니다.

확장 차수

```
pub const EXP_DEGREE: usize = 8; // 확장 차수
```

```
pub(crate) const DEGREE: usize = BASE_DEGREE + EXP_DEGREE; // 총 차수 14
```

Feistel 암호를 이용한 확장 부모 생성

Feistel 키 파생:

```
pub fn derive_feistel_keys(porep_id: PoRepID) -> [u64; 4] {
let mut feistel_keys = [0u64; 4];
let raw_seed = derive_porep_domain_seed(FEISTEL_DST, porep_id);
// 32바이트 시드를 4개의 64비트 키로 분할
feistel_keys[0] = u64::from_le_bytes(raw_seed[0..8].try_into().unwrap());
feistel_keys[1] = u64::from_le_bytes(raw_seed[8..16].try_into().unwrap());
feistel_keys[2] = u64::from_le_bytes(raw_seed[16..24].try_into().unwrap());
feistel_keys[3] = u64::from_le_bytes(raw_seed[24..32].try_into().unwrap());
feistel_keys
}
```

확장 부모 생성 과정:

```
fn correspondent(&self, node: usize, i: usize) -> u32 {
        let a = (node * self.expansion_degree) as feistel::Index + i as feiste
        let transformed = feistel::permute(
                self.size() as feistel::Index * self.expansion_degree as feistel
                a,
                &self.feistel_keys,
```

```
                self.feistel_precomputed,

        );

        transformed  as u32 / self.expansion_degree as u32

}
```

Feistel 순열의 특성

가역성:

- Feistel 암호는 대칭적 구조로 암호화와 복호화가 동일한 과정
- 순방향과 역방향 레이어에서 동일한 부모-자식 관계 유지

균등 분포:

- 전체 검색 공간에서 균등한 부모 노드 분포 보장
- 특정 영역에 편중되지 않는 부모 선택

결정론적 생성:

- 동일한 입력에 대해 항상 동일한 출력 생성
- 증명 검증 시 부모 노드 재생성 가능

보안 특성 및 공격 방어

시간-공간 트레이드오프 공격 방어

순차 계산 강제:

- 즉시 전임자 의존성을 통한 병렬화 방지
- 각 노드는 이전 노드의 계산 완료 후에만 계산 가능

깊이 견고성:

- 일부 데이터 삭제 시에도 전체 재계산 필요
- 저장 공간 절약을 위한 데이터 삭제 공격 무효화

암호학적 가정

해시 함수 보안:

- SHA256, Blake2s, Poseidon의 일방향성 및 충돌 저항성
- 무작위 오라클 모델에서의 보안성 증명

Feistel 순열 보안:

- 3라운드 이상의 Feistel 구조는 의사 무작위 순열
- 키 복구 공격에 대한 저항성

이산 로그 문제:

- BLS12-381 곡선에서의 이산 로그 문제 어려움
- zk-SNARK 증명의 건전성 보장

성능 최적화 기법

해시 함수 최적화

SIMD 명령어 활용:

- Blake2s의 벡터화된 구현
- 병렬 해시 계산을 통한 성능 향상

GPU 가속:

- Poseidon 해시의 CUDA/OpenCL 구현
- 대규모 머클 트리 구축 시 GPU 활용

메모리 접근 최적화

부모 캐시 시스템:

```
pub fn parent_cache(&self) -> Result<ParentCache> {
    let default_cache_size = SETTINGS.sdr_parents_cache_size;
    let cache_entries = self.size() as u32;
    let cache_size = cache_entries.min(default_cache_size);
    ParentCache::new(cache_size, cache_entries, self)
}
```

프리페치 최적화:

```
fn prefetch(parents: &[u32], data: &[u8]) {
    for parent in parents {
        let start = *parent as usize * NODE_SIZE;
        let end = start + NODE_SIZE;
        prefetch!(data[start..end].as_ptr() as *const i8);
    }
}
```

디스크 I/O 최적화

순차 접근 패턴:

- 노드 데이터의 순차적 배치
- 디스크 시크 시간 최소화

레벨별 캐싱:

- 자주 접근하는 머클 트리 상위 레벨 메모리 캐시
- 캐시 히트율 향상을 통한 성능 개선

이러한 암호학적 기초 요소들의 조합을 통해 파일코인은 안전하고 효율적인 저장 증명 시스템을 구현하며, 각 구성 요소는 전체 시스템의 보안성과 성능에 중요한 역할을 담당합니다.